

Convolution as a local operation

Project: Image-pattern Network Investigation

1. Your mission

Your learning target

I can compute a local weighted sum at each valid image position.

I can explain shared weights and predict output dimensions.

I can check how stride boundaries and activation affect the result.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Image tensors and channels. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Debugging shapes and gradients. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Training and validation curves. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Compute a local weighted sum at each valid image position

Explain shared weights and predict output dimensions

Check how stride boundaries and activation affect the result

Retrieve what you already know

Why must an image schema record axis order as well as shape?

My starting explanation

This week's end goal

By the end, I can explain convolution as a local operation using a traced example and a stated limitation.

2. Notice the evidence

Use a fictional single-channel image with rows [1, 2, 0], [0, 1, 3] and [2, 0, 1]. Coordinates are zero-based row then column. Our kernel has rows [1, -1] and [0, 1]. At each position, multiply matching entries of a 2 by 2 window and the kernel, then add the four products. Use bias 0, stride 1 and no padding.

At the top-left window, the sum is 1 times 1 plus 2 times -1 plus 0 times 0 plus 1 times 1 = 0. Slide the same kernel right, then down. Only use windows that fit entirely inside the image; this is the valid boundary rule.

Window start · Four products · Sum

Window start: [0, 0]

Four products: 1, -2, 0, 1

Sum: 0

Window start: [0, 1]

Four products: 2, 0, 0, 3

Sum: 5

Window start: [1, 0]

Four products: 0, -1, 0, 0

Sum: -1

Window start: [1, 1]

Four products: 1, -3, 0, 1

Sum: -1

The feature map has rows [0, 5] and [-1, -1]. This neural-network convention keeps the kernel orientation

fixed; mathematically it is cross-correlation. Mathematical convolution reverses the kernel along both spatial axes. We use the fixed-orientation convention throughout, so every hand calculation and program agrees.

List the given facts and one fact that is not supplied.

3. See how it works

The following function assumes nonempty rectangular numeric arrays and a positive integer stride. It rejects a kernel larger than the image. It computes one input channel, one output map, zero bias and no padding; no activation is applied inside the function.

```
def local_filter(image, kernel, stride=1):
    h, w = len(image), len(image[0])
    kh, kw = len(kernel), len(kernel[0])
    if kh > h or kw > w:
        raise ValueError("Kernel does not fit")
    out = []
    for r in range(0, h - kh + 1, stride):
        row = []
        for c in range(0, w - kw + 1, stride):
            value = sum(image[r+i][c+j] * kernel[i][j]
                        for i in range(kh) for j in range(kw))
            row.append(value)
        out.append(row)
    return out

x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]
assert local_filter(x, k) == [[0, 5], [-1, -1]]
assert local_filter(x, k, 2) == [[0]]
```

The kernel weights are reused at all four positions. There are four kernel parameters, not sixteen separate weights. If we add one shared bias for this output map, there are five parameters. The number of multiply

operations grows with the number of windows even though the parameter count stays fixed.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

How many shared kernel weights does the lesson use? How many products are computed over all four valid windows? What changes if one shared bias is added?

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Add shared bias2 after each patch sum.

Prediction: Feature map becomes $\begin{bmatrix} -2 & -2 \end{bmatrix}$.

Reason: Bias adds2 to each output; it does not change patch products or output shape.

5. Words that unlock the idea

Word · Meaning

Word: Kernel

Meaning: A small array of weights applied to a local input neighbourhood

Word: Feature map

Meaning: The spatial array of outputs produced by applying a filter

Word: Stride

Meaning: The distance between successive window starting positions

Word: Padding

Meaning: Values added around an input boundary under a declared rule

Use two terms to explain a step in this week's example.

6. Follow a worked example

One shared kernel produces four local sums

Window start · Kernel rows · Output value

Window start: Row 0 column 0

Kernel rows: [1 -1] and [0 1]

Output value: 0

Window start: Row 0 column 1

Kernel rows: [1 -1] and [0 1]

Output value: 5

Window start: Row 1 column 0

Kernel rows: [1 -1] and [0 1]

Output value: -1

Window start: Row 1 column 1

Kernel rows: [1 -1] and [0 1]

Output value: -1

Read windows in row order. Each output summarizes one neighbourhood. With stride 1, adjacent windows overlap, but their output values occupy separate cells.

Change only input[0][0] from 1 to 4. Which valid windows include it? Compute the changed feature map and explain why the other three outputs stay the same.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Convolution as a local operation

Draw the exact patch-by-patch arithmetic of a local image operation.

The facts we will use

Image rows [1, 2, 3],[4, 5, 6]; kernel rows [1, 0],[0,-1]. Use unflipped kernel, no padding, stride1, bias0.

All inputs and outcomes in this worked example are invented classroom data; no real model run is claimed.

Image used by both local kernel windows

	col0	col1	col2
row0	1	2	3
row1	4	5	6

Read a cell using both its row and column labels.

Convolution as a local operation

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. First patch

$$1 \times 1 + 2 \times 0 + 4 \times 0 + 5 \times (-1) = -4.$$



2. Second patch

$$2 \times 1 + 3 \times 0 + 5 \times 0 + 6 \times (-1) = -4.$$



3. Feature map

Output shape 1 x 2 and map $\begin{bmatrix} -4 & -4 \end{bmatrix}$; the same kernel is reused at each position.

CHANGE ONE THING

Add shared bias 2 after each patch sum.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. First patch

What happens and why: $1 \times 1 + 2 \times 0 + 4 \times 0 + 5 \times (-1) = -4.$

Step: 2. Second patch

What happens and why: $2 \times 1 + 3 \times 0 + 5 \times 0 + 6 \times (-1) = -4.$

Step: 3. Feature map

What happens and why: Output shape 1×2 and map $\begin{bmatrix} -4 & -4 \end{bmatrix}$; the same kernel is reused at each position.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

A local kernel combines the values in one small image window with a fixed set of weights. Slide that same kernel to the next valid window and repeat the products. Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped kernel; adding bias changes each window result without changing which pixels were used.

What this example does not establish

A local filter detects a numerical pattern; one feature map does not establish recognition or learned visual understanding.

Read the labelled picture: Why is flipping the kernel an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.

Change one thing: Add shared bias2 after each patch sum. Predict the result before checking the explanation. What remains the same?

7. Find and repair a mistake

For no padding and a kernel that fits, output height is $\text{floor}((H - K_h) / s) + 1$ and width is $\text{floor}((W - K_w) / s) + 1$, where s is the positive integer stride. Floor means round down. Count allowed start positions before using the formula. With a 3 by 3 image, 2 by 2 kernel and stride 2, the only row and column start is 0, so the output is 1 by 1.

Padding changes which windows exist and what they contain. Adding one border of zeros on every side changes this 3 by 3 input to 5 by 5. At stride 1 with our 2 by 2 kernel, output shape becomes 4 by 4. Its top-left window is $\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$ and its output is 1. Zero padding is a chosen boundary assumption, not a measured black region outside the picture.

If ReLU is applied after the unpadded sums, the feature map becomes $\begin{bmatrix} 0 & 5 \\ 0 & 0 \end{bmatrix}$. Negative responses are valid weighted sums before activation; they are not necessarily a software error. Keep raw sums available when debugging so ReLU does not hide a mistake.

Locate the earliest unsupported step

For the lesson image and kernel, give the output shape with no padding and stride 2. Then give the shape with one zero border on each side and stride 1, and calculate that padded map's top-left output.

State one repair and the checks you would rerun.

8. Practise together

Use the lesson image and kernel at window start $[0, 1]$. List the window rows, four products and output before activation.

Change only `input[0][0]` from 1 to 4. Which valid windows include it? Compute the changed feature map and explain why the other three outputs stay the same.

Explain your prediction to a partner before comparing results.

Practise with less help: Separate parameters and operations

Use workbook W2 for the connected example “Convolution as a local operation”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Convolution as a local operation. Proposed explanation: “Each window has its own set of learned weights”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Why is flipping the kernel an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

How many shared kernel weights does the lesson use? How many products are computed over all four valid windows? What changes if one shared bias is added?

```
def local_filter(image, kernel, stride=1):  
    h, w = len(image), len(image[0])  
    kh, kw = len(kernel), len(kernel[0])  
    if kh > h or kw > w:  
        raise ValueError("Kernel does not fit")  
    out = []  
    for r in range(0, h - kh + 1, stride):  
        row = []
```

```

    for c in range(0, w - kw + 1, stride):

        value = sum(image[r+i][c+j] * kernel[i][j]

                     for i in range(kh) for j in range(kw))

        row.append(value)

    out.append(row)

return out

x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]

assert local_filter(x, k) == [[0, 5], [-1, -1]]

assert local_filter(x, k, 2) == [[0]]

```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what

did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Use the lesson image and kernel at window start [0, 1]. List the window rows, four products and output before activation.

For the lesson image and kernel, give the output shape with no padding and stride 2. Then give the shape with one zero border on each side and stride 1, and calculate that padded map’s top-left output.

A program reverses both axes of the lesson kernel to [[1, 0], [-1, 1]]. Compute the top-left output. Does it agree with our declared convention? Explain what the test checks.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled convolution as a local operation evidence sheet to Image-pattern Network Investigation. Show how you can compute a local weighted sum at each valid image position. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week’s record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record:

Project evidence: Inputs permitted at prediction time

Record:

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For your Image-pattern Network Investigation, save the input, procedure, observed result and limitation of this check. Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

12. Show what you know

These kernel weights were chosen for arithmetic practice, not fitted from data. A response of 5 is neither a probability nor proof that an object was recognised. A trainable convolutional layer adjusts shared parameters using training examples. Additional channels require summing contributions across channels; that is beyond this single-channel calculation.

A teammate reports a 3 by 3 output for the unpadded lesson image and calls its largest number a probability. Identify the missing or incorrect assumptions and give the expected shape and meaning under our declared setup.

Sharing weights applies the same local rule at each allowed position. It does not guarantee that a whole image classifier ignores object location, especially when strides, boundaries or later layers change the result. Next week studies pooling and the spatial information it can discard.

A program reverses both axes of the lesson kernel to $\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$. Compute the top-left output. Does it agree with our declared convention? Explain what the test checks.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

One shared kernel produces four local sums

Window start · Kernel rows · Output value

Window start: Row 0 column 0

Kernel rows: [1 -1] and [0 1]

Output value: 0

Window start: Row 0 column 1

Kernel rows: [1 -1] and [0 1]

Output value: 5

Window start: Row 1 column 0

Kernel rows: [1 -1] and [0 1]

Output value: -1

Window start: Row 1 column 1

Kernel rows: [1 -1] and [0 1]

Output value: -1

Read windows in row order. Each output summarizes one neighbourhood. With stride 1, adjacent windows overlap, but their output values occupy separate cells.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Filter a new image independently

Use image rows [2, 1, 0], [1, 3, 2] and kernel rows [0, 1], [-1, 2]. Use no padding, stride 1 and bias 0. Give the output shape, both four-product sums and the feature map. Then add a shared bias of -5 and apply ReLU.

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Use image rows [2, 1, 0], [1, 3, 2] and kernel rows [0, 1], [-1, 2]. Use no padding, stride 1 and bias 0. Give the output shape, both four-product sums and the feature map. Then add a shared bias of -5 and apply ReLU.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.

15. Your lesson route

Start with this question: Draw the exact patch-by-patch arithmetic of a local image operation.

By the end, I can · How I will show it

By the end, I can: Compute a local weighted sum at each valid image position

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Explain shared weights and predict output dimensions

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

By the end, I can: Check how stride boundaries and activation affect the result

How I will show it: Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project evidence. Your teacher may spread this lesson across several classes.

Keep your first prediction. Compare it with what the evidence shows and explain any change.

16. Finish and return

Close your notes. Explain this goal in your own words: Compute a local weighted sum at each valid image position

My deciding step, evidence and one remaining question

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an input or assumption and justify your prediction.

Save your workbook evidence. At the next review, try a different case before opening your old answer.

17. Supported practice, repair and portfolio

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Draw the exact patch-by-patch arithmetic of a local image operation.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? Output shape 1×2 and map $[-4, -4]$; the same kernel is reused at each position.

1. Feature map
2. First patch
3. Second patch

My selected conclusion, deciding fact and explanation

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

After your attempt, compare with the supported explanation: Feature map Output shape 1×2 and map $[-4, -4]$; the same kernel is reused at each position. A local kernel combines the values in one small image window with a fixed set of weights. Slide that same kernel to the next valid window and repeat the products. Sharing weights lets the operation look for the same local pattern at different positions. This

example declares an unflipped kernel; adding bias changes each window result without changing which pixels were used.

Practice 2: explain the deciding evidence

Changed-case practice: Add shared bias2 after each patch sum. Which conclusion is supported by the supplied facts?

1. We can decide the result without examining the changed input or the procedure.
2. Feature map becomes $\begin{bmatrix} -2 & -2 \end{bmatrix}$.
3. This one example guarantees correct results for every future input.

My selected conclusion, deciding fact and explanation

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

After your attempt, compare with the supported explanation: Feature map becomes $\begin{bmatrix} -2 & -2 \end{bmatrix}$. Bias adds2 to each output; it does not change patch products or output shape.

Repair a missing building block

Review Image tensors and channels (ai-5/20). Goal: Read a pixel using declared row column and channel axes. Short practice: Where is the bottom green pixel in the first batched image? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.. Return to this week and explain what you changed.

Review Debugging shapes and gradients (ai-5/18). Goal: Check dimension contracts before computing a layer. Short practice: What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.. Return to this week and explain what you changed.

Review Training and validation curves (ai-5/15). Goal: Compute comparable training and validation losses at saved checkpoints. Short practice: Can the original epoch 3 training loss 1 justify claiming best generalisation? Point to the relevant step in the picture, then write the changed result and the rule or evidence that supports it.. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual output or error, change one input, then run again. Keep code, predictions, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run evidence. Open the HTML copy to read or print it. Drafts stay in this browser when storage is available; clear drafts on a shared device.